GENERATIVE AND AGENTIC AI · TOPIC 7

Agentic AI

From Tool-Using LLMs to Agentic Harnesses

Laura Wynter, PhD
SMU School of Computing and Information Sciences
IS469 · AY2026–27

ROADMAP

What we'll cover

01 Tool use & function calling

02 The agent loop & harness anatomy

03 MCP & agent protocols

04 Planning & reasoning patterns

05 Memory & context engineering

06 Multi-agent orchestration

07 Agent Skills

08 Evaluating & observing agents

09 Safety, guardrails & permissions

10 Applications & what's next

FRAMING

Where we are

- So far: prompting, fine-tuning (T3–4), serving (T5), evaluation (T6), all one-shot uses of the LLM
- All of that still leaves the LLM as a one-shot function: prompt in → text out
- An agent lets the LLM take actions, observe results, and keep going toward a goal
- Today's thesis: an agent = an LLM (the reasoning) + a harness (the scaffolding around it)
- The harness is where most real-world agent quality is won or lost

FRAMING

A bare LLM call vs. an agent

AI Agent is LLM (transformer) with a harness around it

Bare LLM call

- One prompt in, one response out
- No access to live data or tools
- Cannot act on the world
- Knowledge frozen at training time
- You orchestrate every step by hand

Agent = LLM + harness

- Runs a loop over multiple steps
- Calls tools and observes real results
- Acts: search, run code, call APIs, send email
- Pulls fresh context on demand
- Plans and self-corrects toward a goal

FRAMING

The 2026 agentic landscape

no visual level, convert to text first

- Agentic products are the main event: coding agents (Claude Code, Codex), workplace and enterprise agents
- Computer-use agents drive a real GUI by screenshot-and-click
  - …but the least reliable modality: strong on benchmarks, still fails most real multi-step workflows
- "Agentic task completion", long-horizon autonomous work, is the frontier
- METR (Model Evaluation & Threat Research, an independent AI-evaluation lab) tracks the length of task an agent can complete: it doubles roughly every 7 months
- Agent benchmarks are headline metrics: SWE-bench Verified, Terminal-Bench 2.0, OSWorld
- Tools and data now connect through a shared standard protocol rather than one-off integrations (§3)
- The open question: autonomy vs. control and safety

01

Tool Use & Function Calling

First, what a tool actually is: the only way an LLM reaches anything outside its own text.

Anatomy of a tool call

API example : Web Search (Google API) acess to LLM to do websearch for you

- A tool is just a FUNCTION the LLM is allowed to ask for by name, it never runs it itself
- It is declared with a schema: name, natural-language description, and typed arguments (each argument has a declared type, e.g. string, integer, date)
- The description is entered into the LLM as part of its prompt; the LLM reads it to decide when and how to call the tool

LLM generates the text to call the tool

- Flow: LLM emits a structured call → harness checks the argument types & executes → result returned as an observation
- The LLM never runs code itself, the harness does, and controls what's exposed

The life of a single tool call



The LLM only chooses the call, the harness validates, executes, and controls what's exposed.

Designing the tool interface the LLM sees

Wrapper is what LLM needs to generate text to call the tool

- A tool usually wraps something that already exists (an API, a database query, a script); 'designing' it means writing the wrapper the LLM sees: name, description, argument schema, and what it returns
- Clear, specific descriptions, ambiguity causes wrong or missing calls
- Typed, validated arguments; return errors the LLM can recover from (not stack traces)

if tool call fail base on error in result

- Keep tools focused, and safe to call twice: repeating the same call must not cause a second side effect (e.g. a second payment)

input into LLM , when running inference . Size of context window is limited by memory

- Return concise, structured results, don't flood the context window
- Common failure mode: a vague tool the LLM calls at the wrong time, or can't recover from

Parallel calls & tool-call traces

- Modern LLMs can issue several tool calls in one step (parallel), faster, but harder to attribute
- Every tool call is a logged event: name, arguments, result, latency, success/failure
- These traces are first-class data, for debugging AND for evaluation (we'll quantify them in §8)
- If you can't see the tool calls, you can't tell whether the agent is actually working

Code execution as a universal tool

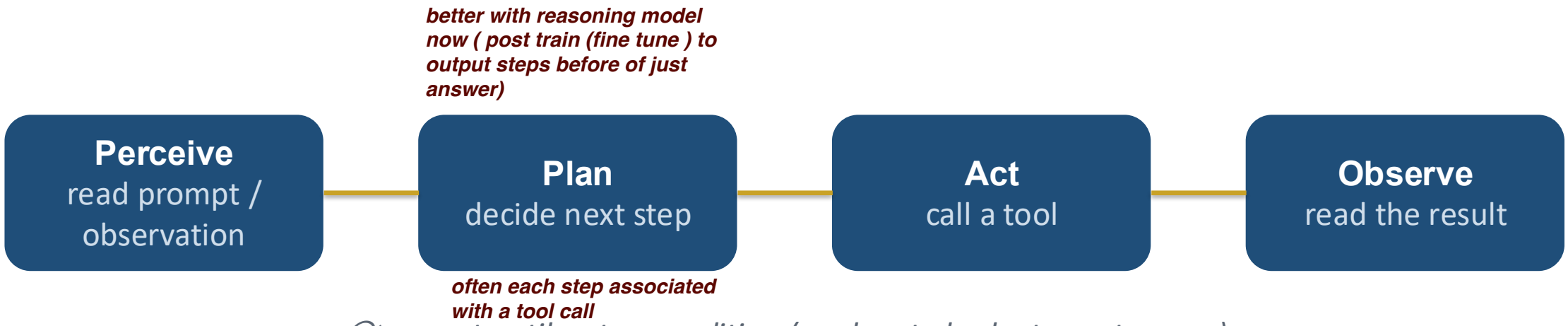
- Giving the agent a sandboxed code interpreter is often more powerful than many narrow tools
- Handles arithmetic, data wrangling, plotting, file manipulation, deterministically
- Trade-off: code execution is the highest-risk tool → must be sandboxed (see §9)
- Coding agents (Claude Code, Codex) are this pattern, industrialised

02

The Agent Loop & Harness Anatomy

An agent is an LLM wrapped in a runtime. Let's open up that runtime.

The core agent loop



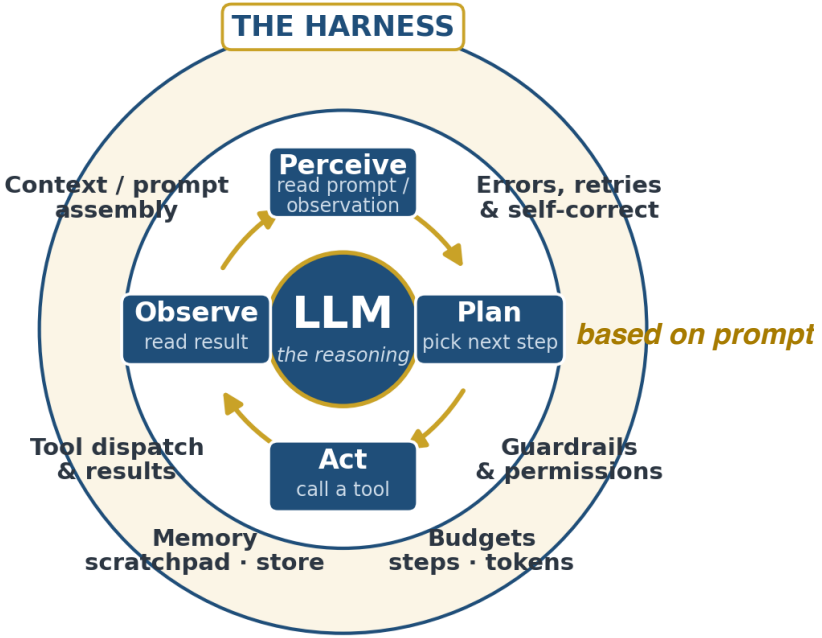
LLM decides: Plan + which tool to Act with. **Harness does everything else:** assemble context, run the tool, parse output, manage memory, enforce budgets.

What the harness is responsible for

- The LLM only chooses the next step, the harness does everything around it:
  - Context / prompt assembly and output parsing
  - Tool dispatch and result handling
  - Memory, scratchpad, running history, long-term store
  - Error handling, retries and self-correction
  - Step- and token-budgets, and the stop condition
  - Permissions and sandboxing (what the agent is allowed to do)
- Same LLM + a better harness = a large capability difference (see: coding agents)

The LLM decides; the harness ring does the rest

The agent loop: an LLM inside a harness ring (schematic)



The LLM decides Plan + Act; the harness ring does everything else, and repeats the cycle until a stop condition.

Schematic: the perceive → plan → act → observe cycle turns around the LLM, wrapped in a harness that handles context, tools, memory, budgets and guardrails, and repeats until a stop condition.

Building a minimal harness, the loop itself

harness is managing call to the LLM as well as excuting the tool calls

- A harness is not magic, its core is a short loop you can write yourself:

some info here stored in vector database, not just in loop. These are in embedding form as they come from an LLM

prompt (content of message)

```
messages = [system_prompt, user_task]
for step in range(max_steps):          # the step budget
    reply = llm(messages, tools)       # LLM decides
    if reply.tool_call:                # act
        obs = run_tool(reply.name, reply.args)   # harness executes
        messages.append(observation(obs))        # observe → loop
    else: return reply.text            # stop: final answer
```

tools is a list it has access to

if is reasoning model, the reply may have a lot of things

how to add budget on retries?
- uk it fails when obs return error when tool is ran

assume output iis structure with tool_call

retries in observation function
and also critique function

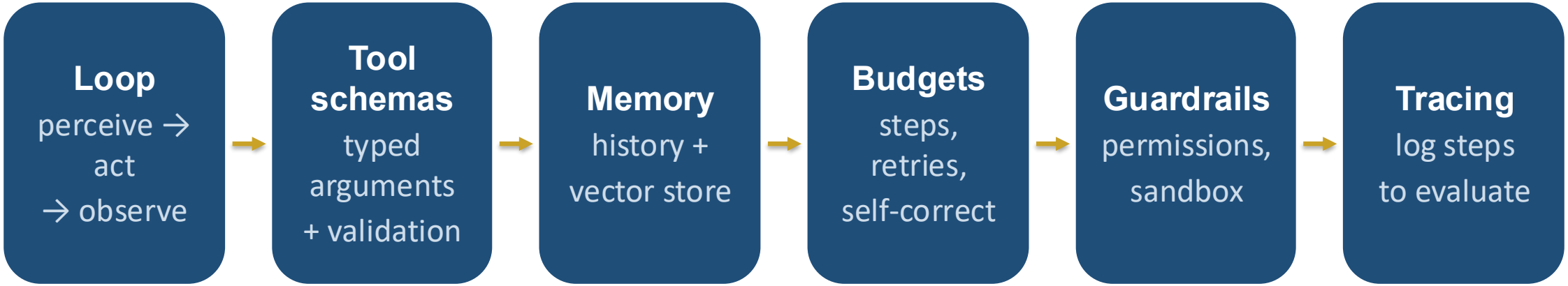
what is memory here?
- messages (store output of the tool run) : original prompt + output of too call

- That loop IS the harness, everything else is hardening built around it

From a loop to a real harness

tool calling is executing code —> introduce security vulnerability, use sandbox which is env with no full acess & restrictive permission

embedding- a vector representation of tokens



Add one layer at a time, exactly the progression you build in this topic's hands-on lab.

can we have embedding for the whole paragraph? -pooling

ReAct: the canonical loop

- ReAct = interleave Reasoning and Acting in one trace
  - Thought: what should I do next, and why
  - Action: a tool call with arguments
  - Observation: the tool's result, fed back into context
- The loop repeats; each observation updates the LLM's working context
- Simple, transparent, and still the default backbone inside most agent frameworks

ReAct: reasoning and acting in one growing trace



Schematic: Thought → Action → Observation blocks accumulate in a single growing context (the message history) and loop until the goal is met, an agent is a loop over a growing trace, not one call.

Ways an agent takes action

Agent type	How it acts	Best for
JSON / tool-calling	Emits a structured tool call (name + args)	The default for most agents
Code agent	Writes a code block the harness runs	Data, logic, general computation
Function-calling API	Selects an API + arguments to invoke	Calling existing services
Computer-use / GUI	Perceives via screenshot, acts via mouse + keyboard	Apps with no API, click through a UI

Modern LLMs are trained for native tool-calling; computer-use adds a screenshot-and-click modality for software with no API.

03

MCP & Agent Protocols

Standard plumbing so agents reach tools and data without bespoke glue.

MCP, the Model Context Protocol

make syntax required for api consistents

- Without a standard: every LLM × every tool = its own custom integration (N × M glue)
- MCP is an open protocol, think "USB-C for tools and data", introduced by Anthropic, now widely adopted
- An MCP server exposes capabilities; any MCP-aware host/agent can use them
- Write the integration once; reuse it across agents and LLMs

How MCP is structured

MCP server run somwhere, client is agent calling it

- Host / client: the app or agent runtime (e.g. an IDE, Claude Desktop, your agent)
- MCP server: a process that exposes three things:
  - Tools: actions the agent can invoke
  - Resources: data/context the agent can read
  - Prompts: reusable prompt templates
- Client and server speak a standard JSON-RPC-style protocol over a transport

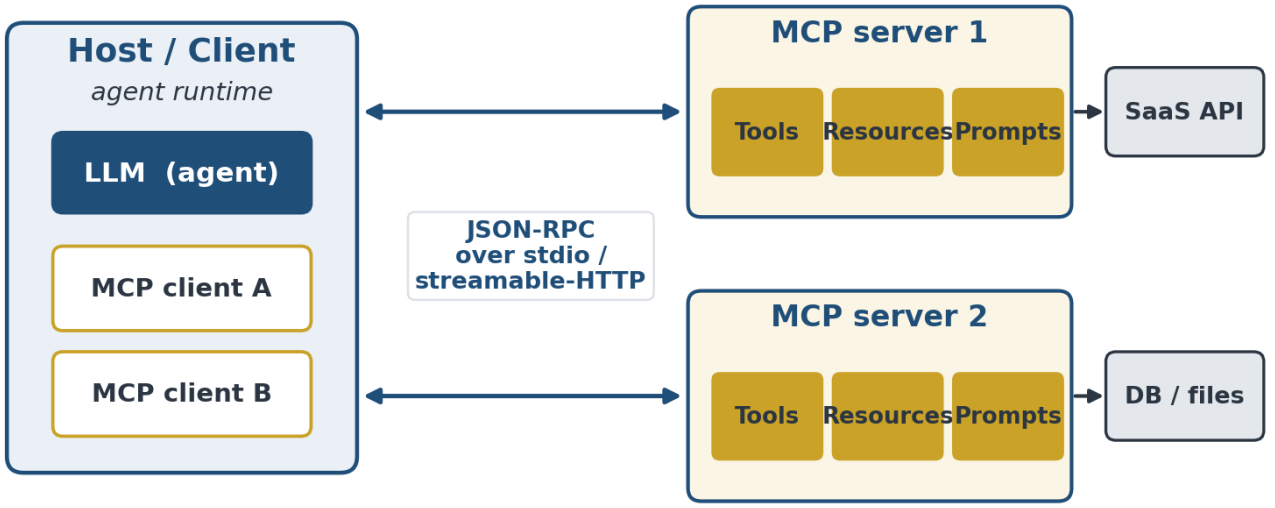
MCP architecture at a glance



Write the integration once as an MCP server; any MCP-aware agent reuses it, "USB-C for tools & data".

MCP: one host, many reusable servers

MCP architecture: one host, many reusable servers (schematic)



Write the integration once as an MCP server; any MCP-aware host reuses it , "USB-C for tools & data".

Schematic: a host / agent runtime with MCP clients talks over a transport (stdio / streamable-HTTP) to MCP servers, each exposing Tools, Resources and Prompts and wrapping a real backend.

How client and server talk: transports

not in quiz

- MCP messages are JSON-RPC; they travel over a transport, two matter in practice:
  - stdio, the server runs as a local subprocess the host launches (Claude Desktop, Cursor, our lab's reference server)
  - Streamable HTTP, for remote servers; it replaced the older HTTP + SSE two-endpoint design
- Streamable HTTP can run stateful or stateless (next slide), and that choice drives how you deploy

Stateful vs. stateless, what the words mean

- State = what a server remembers about you between requests: who you are, what you asked before, where you are in a session
- Stateful: like a phone call. A session is opened, the server keeps notes on it, and every later message must reach the SAME server
- Stateless: like sending a letter. Each request carries everything needed to answer it; the server remembers nothing afterwards
- Stateless MCP: every tool call is one self-contained HTTP request, no session to open, no session id to carry, nothing for the server to keep
- Why it matters: a stateless server can be copied at will and any copy can answer any request, so it scales out and runs serverless

Stateful vs. stateless MCP

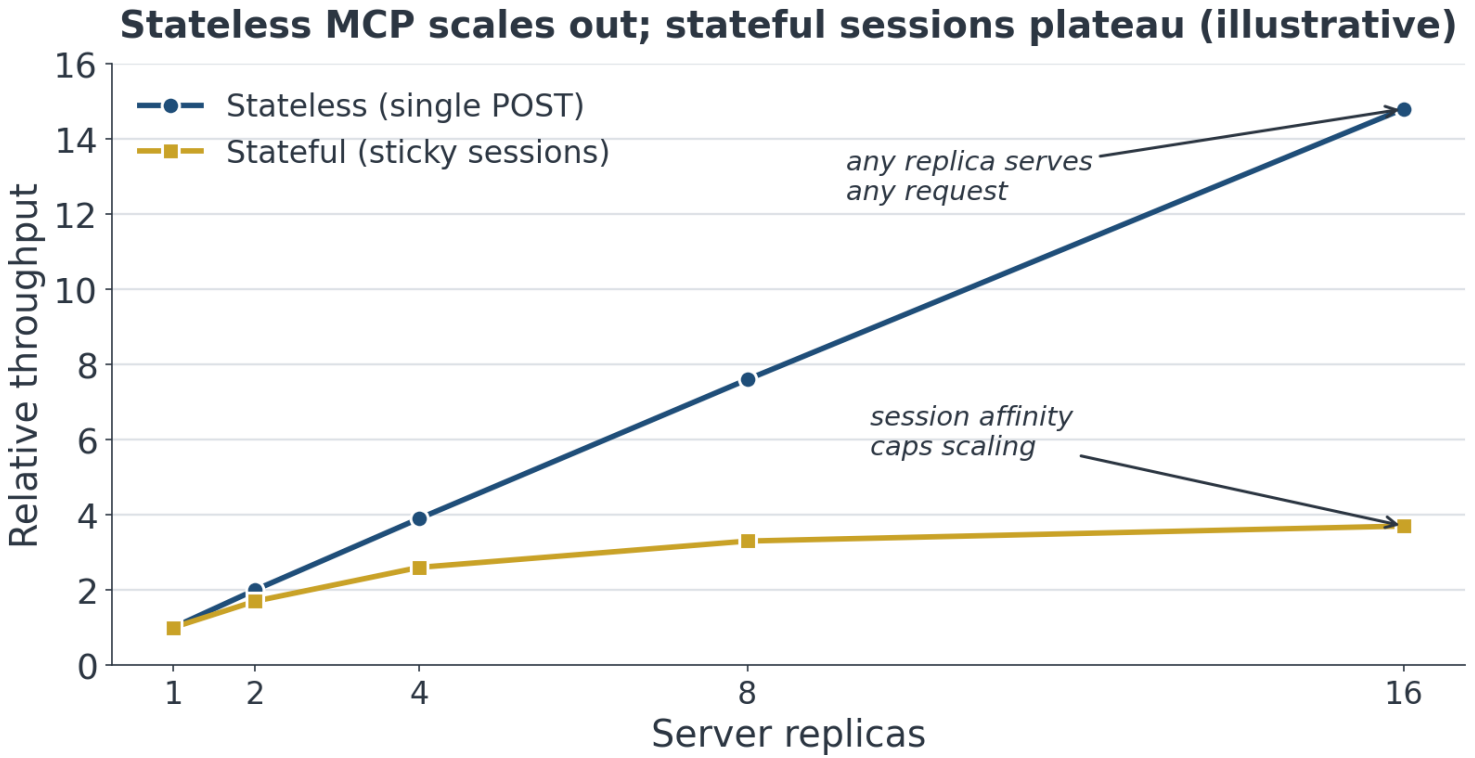
Stateful (the original way)

- Init handshake first → server returns an Mcp-Session-Id
- Every later call must carry that session id
- The load balancer must pin you to the SAME backend (sticky routing)
- Awkward to run serverless or scale horizontally

Stateless (the 2026 shift)

- One HTTP POST does everything, no session id, no handshake
- Server keeps no per-client state between requests
- Any replica can serve any request → serverless & auto-scaling
- Cleaner to implement on both client and server

Stateless servers scale out; sticky sessions don't

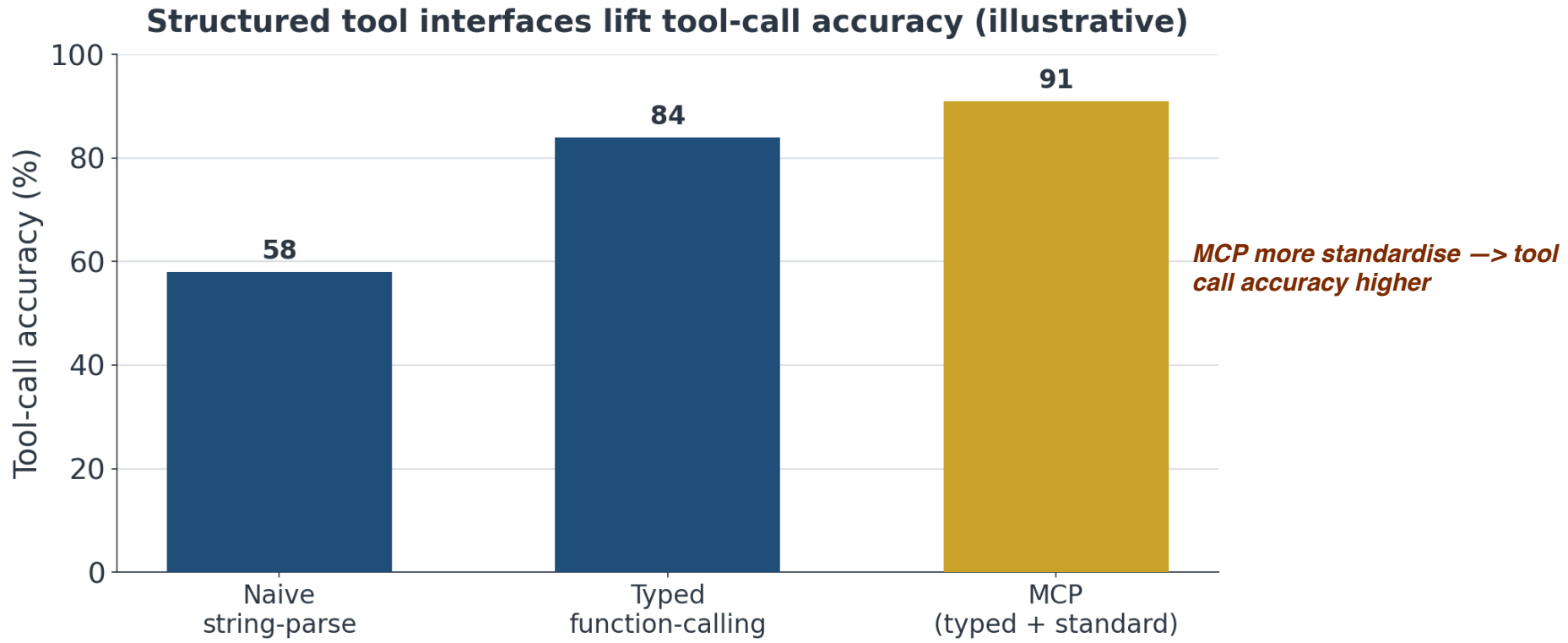


Illustrative: with no session to pin, any replica serves any request, so throughput grows with replicas instead of plateauing.

Why MCP is interesting again

- MCP surged in 2025, then looked eclipsed: an agent with a terminal and curl is more flexible
- But arbitrary shell access is risky and needs a powerful LLM to use safely
- Stateless MCP rebalances it: bounded, auditable tools that even small local LLMs can drive
- Plus it now deploys serverless and scales cleanly, the old session-affinity pain is gone
- Takeaway: MCP's edge is safety + reuse + scale, not raw flexibility (per Simon Willison, 2026)

Structured tool interfaces lift tool-call accuracy



Illustrative: hand-parsing free text is error-prone; native typed function-calling helps; MCP adds typed, standardised tools any agent can drive reliably, accuracy climbs as the interface gets more structured.

04

Planning & Reasoning Patterns

Beyond react-one-step-at-a-time: how agents decide what to do.

Reasoning models: LLMs that think first

model that explain steps to solve a task before outputting solution

When we pretrain the LLM, output doesnt look like this -> objective is to predict the next word

Good for Agentic AI : good way as each step is associated with the tool.

- Some LLMs now "think" before they answer, working through the problem step by step, internally, first
- That hidden working-out is chain-of-thought reasoning; the LLM can catch and fix its own mistakes
- They spend more compute at answer time on harder questions, called test-time compute
- The payoff: much stronger on multi-step problems, maths, coding, planning, logic
- You've likely used them: the "thinking" / "reasoning" modes in ChatGPT, Claude, Gemini and DeepSeek

Standard LLM vs. reasoning model

Standard LLM

- Answers immediately, in one pass
- Fast and cheap
- Good for simple, direct questions
- Can stumble on multi-step problems

Reasoning model

- Thinks step by step first, then answers
- Slower and costs more (thinking tokens)
- Much stronger on hard, multi-step tasks
- Can plan, check itself, and backtrack

Why reasoning models matter for agents

- An agent is only as good as the "brain" deciding each step of its loop
- Reasoning models plan better, catch their own errors, and recover, exactly what an agent needs
- Trade-off: thinking is slower and pricier → use it for the hard steps, a cheaper LLM for routine ones
- Common pattern: a reasoning model to plan, a fast LLM to carry out each step

vs agile/iterative

ReAct vs. Plan-and-Execute

use reasoning model & just act

- ReAct: decide the next single step, act, observe, repeat, flexible, adapts as it goes
- Plan-and-Execute: draft a full plan first, then execute the steps
  - Fewer LLM calls, more coherent on long tasks
  - But a bad up-front plan is costly, usually paired with re-planning
- Rule of thumb: react for open-ended/interactive; plan for well-structured multi-step tasks

Reflexion & self-critique

- Add a step where the agent critiques its own output before continuing or finishing
- Reflexion: the agent reflects on failures and carries lessons into the next attempt
- Verify-then-continue: check tool results / intermediate answers against a rubric or test
- Cheap and high-impact, catches many errors a single forward pass would ship

adding in a seperate agent to evaluate what have been done so far (critique and add accordingly)

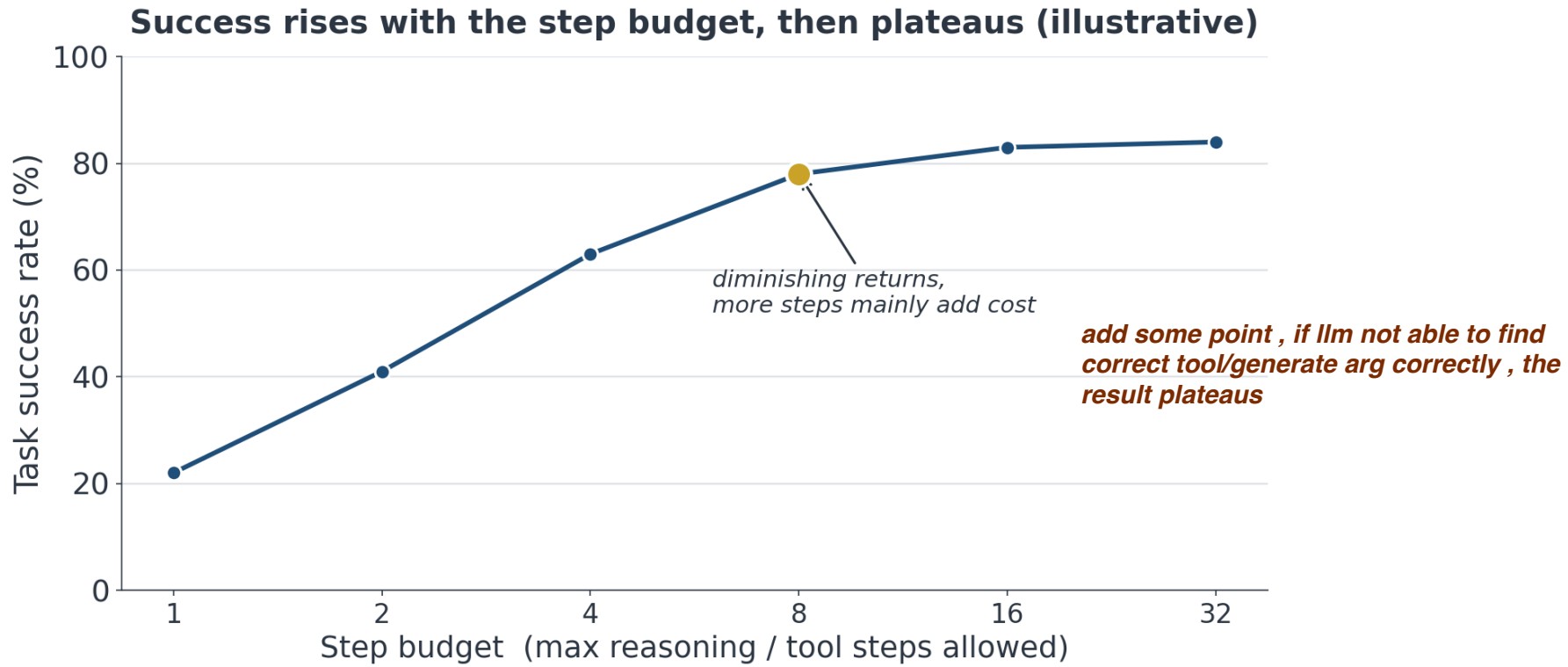
A ladder of planning patterns

iterative



Increasing commitment before acting, from reactive, to planned, to self-correcting.

More steps help, until they don't



Illustrative: too small a step budget and the agent can't finish; success rises as it's allowed more reasoning / tool steps, then plateaus, beyond the knee, extra steps mostly add cost and latency.

Search over actions

- Instead of one trajectory, explore several and select the best
- Tree/graph-of-thought: branch on candidate steps, score, prune
- Best-of-N trajectories with a judge/verifier chooses the winning path
- More compute per task, justified when correctness matters more than latency/cost

Reasoning & effort modes as a cost dial

chain of thoughts: token LLM is generating (not a different kind of token) , fine tuned to generate these reasons as tokens before generating the answer

- Reasoning models 'think' in hidden tokens before answering, better on hard steps, but you pay for those tokens

amount of reasoning is hyparameter at inference (e.g what is the reasoning level)

reasoning level—> how many tokens allocated to the reasoning step

- 2026 LLMs expose an effort / thinking dial (low / medium / high), a direct quality-vs-cost lever
- In an agent, reasoning cost compounds over many steps, budget it deliberately
- Connects to the cost-optimisation ideas from the deployment topic (caching, batching)

Reasoning effort as a cost dial

Effort mode	Thinking	Use when
Low	Few / no hidden tokens	Simple, routine steps
Medium	Moderate reasoning	Most tasks
High	Extensive reasoning	Hard, high-stakes steps

A direct quality-vs-cost lever, and reasoning cost compounds over many agent steps.

05

Memory & Context Engineering

What the agent knows at each step is a design choice, not an accident.

Types of agent memory

input into the LLM which is increasingly appended

- Working memory: the context window, everything the LLM sees right now

storing more than just what give back to the LLM, things useful but may not necessary need to give back to LLM

can be variable, list

- Short-term: a scratchpad / running history within the current task
- Long-term: a store the agent can write to and retrieve from later (often a vector store, see Topic 8)

long term memory in db during the loop

during function, observation can add code to store in db for long term memory

- Memory + reflection: periodically summarise and store what matters for future steps

Three tiers of agent memory



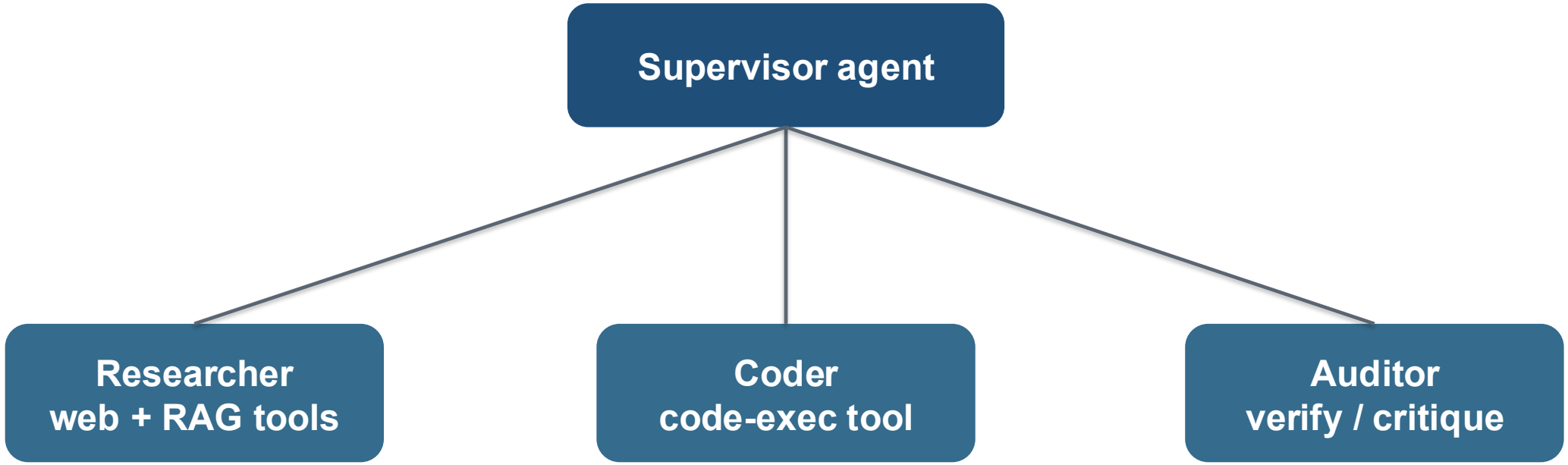
What the agent sees at each step is a design choice, promote what matters, drop the rest.

06

Multi-Agent Orchestration

When one agent isn't enough, and when reaching for many is a mistake.

The supervisor pattern



Supervisor routes each step to a worker; workers return results; supervisor decides when done.

Orchestrating with LangGraph

not a tree, but graph
due to feedback loops

- Model the flow as a state graph: nodes = agents/steps, edges = control flow, a shared state object
- Each node reads the state, does its work, and updates the state for the next node
- Supports linear, branching, and looping flows, and human-in-the-loop checkpoints
- We'll build one in the lab: a supervisor routing to specialist worker agents

§6 · JUDGEMENT

Patterns, and when NOT to use an agent

- Common patterns: supervisor/worker, pipeline/chaining, debate, role pairs (analyst + auditor)
- Ask first: does this decision actually need an LLM?
- If the routing is known, a deterministic if/flag is cheaper, faster, and more reliable than an agent
- Reserve agentic flexibility for genuinely open-ended decisions, not for a switch you could hard-code

07

Agent Skills

Packaging capabilities so agents stay lean and load expertise on demand.

From agent bloat to skills

- As you add tools, instructions and context, the agent's prompt bloats, slower, costlier, more confused
- Agent Skills = modular, portable packages of capability the agent loads only when relevant
- A skill is just a folder: a SKILL.md (name + description + instructions) plus optional scripts / resources
- Interoperable and token-efficient, the agent pulls in a skill's detail on demand
- SKILL.md is now an open, cross-agent format, the same skill runs across Claude Code, Codex CLI, Cursor, Gemini CLI, with skill marketplaces emerging

SKILL.md & progressive disclosure

- Progressive disclosure, the agent sees only as much as it needs, in three levels:
  - Level 1: metadata (name + one-line description), always visible, cheap
  - Level 2: full instructions, loaded when the skill is triggered
  - Level 3: external scripts / references, pulled only if actually needed
- Keeps the context lean while making deep capability available just-in-time

08

Evaluating & Observing Agents

The part most teams skip, and the part that separates a demo from a system.

Agent evaluation ≠ output evaluation

- A right final answer can come from a broken process, and a good process can still fail
- Evaluate the trajectory: the sequence of steps, tool calls and decisions, not just the last message
- This is exactly what final projects get docked for: "it worked in the demo" with no measured evidence
- If agents are in your project, you must measure how well the agentic part worked

What to measure

- Task success / completion rate on a held-out set of tasks
- Tool-call accuracy: right tool, right arguments, recovered from errors
- Step efficiency: did it reach the goal without wandering?
- Cost & latency per task; robustness under input variation
- Mix quantitative metrics with qualitative inspection of real trajectories

§8 · METRICS

The agent evaluation scorecard

Metric	What it measures
Task success	Did it complete the task correctly, on a held-out set?
Tool-call accuracy	Right tool, right arguments, recovered from errors
Step efficiency	Reached the goal without wandering or looping
Cost & latency	Tokens and wall-clock time per task
Robustness	Stable when the input is varied

Evaluate the trajectory, not just the final answer, a right answer can come from a broken process.

Observability & tracing

- You can't evaluate what you can't see, instrument the agent to log every step and tool call
- Tracing tools (e.g. LangSmith) record the full trajectory: inputs, tool calls, outputs, latency, cost
- Turn traces into statistics: success rates, failure clusters, where the flow breaks
- Then act on it, change the flow, the tools, or the prompts, and re-measure

09

Safety, Guardrails & Permissions

An agent that can act can act wrongly. Safety is part of the build.

The new attack surface

- Prompt injection via tools/data: a malicious document or web page instructs the agent to misbehave
- Over-permissioned agents: given more access (files, email, shell) than the task needs
- Runaway loops: an agent that never stops, or repeats a harmful action
- Untrusted MCP servers / tools: the tool itself may be malicious

Controls that make agents safe to run

Control	What it prevents
Sandboxing	Code / tools escaping to the host or network
Human-in-the-loop	Unapproved high-impact actions (send, delete, pay)
Permissions / allow-lists	Access beyond what the task needs
Budgets	Runaway loops burning steps / tokens
Guardrail checks	Injection, PII leaks, policy violations

Safety is a design layer, not an afterthought, these controls are part of the build.

10

Applications & What's Next

Where these pieces show up in real systems.

Case studies, mapped to the harness

- Coding agents (Claude Code, Codex): the loop + tools + permissions, industrialised
- Workplace agents (ChatGPT Work, Claude Cowork): gather context across connected apps, produce work
- Enterprise workflow (watsonx Orchestrate): many connectors, role-based agents
- Domain multi-agent (e.g. IBM's Wimbledon Match Chat): style-tuned, multi-agent
- Each is just the components from today, composed

§10 · JUDGEMENT

Is this actually agentic?

- has access to tools
- have loop (graph) : look at output and come back to it differently
- memory to carry out the loop

- Teardown test: a single frontier-API call behind a nice UI is NOT an agent
  - No loop, no tools, no memory, no evaluation → it's just a prompt
- Ask of any 'agent': where is the loop, the tools, the memory, and the evidence it works?
- Next topic: the single most useful tool you can give this agent, retrieval over your own data (Topic 8)

Key takeaways

01

Agent = LLM + harness

The loop, tools, memory, budgets and guardrails are where capability is won or lost.

02

Tools & MCP

Native tool-calling with typed schemas; MCP standardises how agents reach tools and data.

03

Evaluate the trajectory

Judge the path, not just the answer, tool-call success, steps, cost, completion.

04

Safety is a design layer

Sandboxing, approvals, permissions and budgets are part of the build, not an afterthought.

LAB

Hands-on lab & what's expected

- Progressive build: bare agent loop → typed tools → an MCP tool → memory → LangGraph → trace & evaluate
- You'll finish by measuring your agent: tool-call success and task completion on a small task set
- You fill in the key steps yourself, with a worked solution to check against
- Project expectation: if you use agents, log and analyse the tool-call traces and flow success, not just a final metric